

СОФИЙСКИ УНИВЕРСИТЕТ
„СВ. КЛИМЕНТ ОХРИДСКИ“



ФАКУЛТЕТ ПО МАТЕМАТИКА
И ИНФОРМАТИКА

ДЪРЖАВЕН ИЗПИТ

ЗА ПОЛУЧАВАНЕ НА ОКС „БАКАЛАВЪР ПО СОФТУЕРНО ИНЖЕНЕРСТВО“

ЧАСТ I (ПРАКТИЧЕСКИ ЗАДАЧИ)

Драги абсолвенти:

- Попълнете факултетния си номер в горния десен ъгъл на всички листове.
- Пишете само на предоставените листове, без да ги разкопчавате.
- Решението на една задача трябва да бъде на същия лист, на който е и нейното условие (т.е. може да пишете отпред и отзад на листа със задачата, но не и на лист на друга задача).
- Ако имате нужда от допълнителен лист, можете да поискате от квесторите.
- На един лист не може да има едновременно и чернова, и белова.
- Черновите трябва да се маркират, като най-отгоре на листа напишете „ЧЕРНОВА“.
- Ако решението на една задача не се побира на нейния лист, трябва да поискате нов бял лист от квесторите. Той трябва да се защити с телбод към листа със задачата.
- Всеки от допълнителните листове (белова или чернова) трябва да се надпише най-отгоре с вашия факултетен номер.
- Черновите също се предават и се защитават в края на работата.
- Времето за работа по изпита е 3 часа.

Изпитната комисия ви пожелава успешна работа!

Задача 1.

Задачата да се реши на езика C++. Реализирана е система за управление на автономни роботи – наземни роботи и летящи дронове, използвани при различни типове мисии – патрулиране (Patrol), наблюдение (Surveillance), изследване (Exploration) и спасителни (Rescue).

- Всеки робот притежава уникален идентификатор и текущо ниво на батерията.
- Наземните роботи се използват за придвижване и работа върху различни типове терен. За тях се пази информация за максималното ниво на трудност на терена (terrainLevel). Дроновете се използват за въздушно наблюдение и имат максимална височина на полет (maxAltitude).
- Всяка мисия има тип и ниво на трудност. Значението на трудността зависи от типа на мисията. При Patrol тя описва трудността на терена. При Surveillance описва необходимата височина за наблюдение, а при Rescue описва сложността на спасителната операция.
- Не всички роботи могат да изпълняват всички мисии. Наземните роботи могат да участват в мисии от тип Patrol и Rescue. Дроновете могат да участват в мисии от тип Surveillance и Rescue. Даден робот може да изпълни мисия само ако неговите технически характеристики са достатъчни за съответната трудност на мисията и ако разполага с необходимия заряд на батерията.
- При изпълнение на мисия батерията на робота намалява с количество, равно на два пъти трудността на задачата.
- Мисиите могат да се възлагат на флотилии от роботи, като флотилията е отговорна за поддържането на група от работи и предаването на мисии към тях.

Да се попълнят празните места в дадения по-долу програмен фрагмент в съответствие с условието на задачата, като се осигури коректно управление на паметта за вложените обекти. За улеснение приемете, че при заделянето на обекти с new винаги има достатъчно памет.

```
enum class MissionType { Patrol, Surveillance, Exploration, Rescue };
struct Mission { _____; _____ };
class Robot {
protected:
    std::string id;
    unsigned battery;
public:
    Robot(const std::string& id, unsigned battery) : id(id), battery(battery) {}
    _____ ~Robot() _____;
    const std::string& getId() const { return id; }
    unsigned getBattery() const { return battery; }
    _____ bool canExecute(const Mission& m) const _____;
    _____ clone() const _____;
    void print(std::ostream& os) const {
        os << " [id=" << id << ", battery=" << battery << "]; }
    bool assign(const Mission& m) {
        if (!canExecute(m)) return false;
        battery -= _____;
        return true; }
};
class GroundRobot : public Robot {
    unsigned terrainLevel;
public:
    GroundRobot(const std::string& id, unsigned battery, unsigned terrainLevel)
        : _____, _____ {}
```

```
bool canExecute(const Mission& m) const {
    return battery >= 2 * m.difficulty &&
        (m.type == MissionType::Patrol || m.type == MissionType::Rescue) &&
        terrainLevel >= m.difficulty; }
    _____ clone() const { return _____; }
};

class Drone : public Robot {
    unsigned maxAltitude;
public:
    Drone(const std::string& id, unsigned battery, unsigned maxAltitude)
        : Robot(id, battery), maxAltitude(maxAltitude) {}
    bool canExecute(const Mission& m) const {
        return _____
            _____
            _____; }
    _____ clone() const { return _____; }
};

class Fleet {
    std::vector<_____> robots;
public:
    Fleet() = default;
    ~Fleet() {
        for (Robot* r : robots) _____; }
    Fleet(const Fleet& other) {
        for (const Robot* r : other.robots) robots.push_back(_____); }
    Fleet& operator=(const Fleet& other) {
        if (_____) {
            for (Robot* r : robots) delete r;
            robots.clear();
            for (const Robot* r : other.robots) robots.push_back(_____);
        }
        return _____; }
    _____ add(const Robot& r) { robots.push_back(_____); return *this; }
    void printAll(std::ostream& os) const {
        for (const Robot* r : robots) { r->print(os); os << "\n"; } }
    int assignToAll(const Mission& m) {
        int count = 0;
        for (Robot* r : robots) { if (_____) ++count; }
        return count;
    }
};

int main() {
    Fleet fleet;
    fleet.add(GroundRobot("G1", 100, 5))
        .add(Drone("D1", 100, 8))
        .printAll(std::cout);
    std::cout << fleet.assignToAll({MissionType::Rescue, 4}) << "\n";
    return 0;
}
```

Примерно решение

```
enum class MissionType { Patrol, Surveillance, Exploration, Rescue };
struct Mission { MissionType type; unsigned difficulty; };
class Robot {
protected:
    std::string id;
    unsigned battery;
public:
    Robot(const std::string& id, unsigned battery) : id(id), battery(battery) {}
    virtual ~Robot() = default;
    const std::string& getId() const { return id; }
    unsigned getBattery() const { return battery; }
    virtual bool canExecute(const Mission& m) const = 0;
    virtual Robot* clone() const = 0;
    void print(std::ostream& os) const {
        os << " [id=" << id << ", battery=" << battery << "]; }
    bool assign(const Mission& m) {
        if (!canExecute(m)) return false;
        battery -= 2 * m.difficulty;
        return true; }
};

class GroundRobot : public Robot {
    unsigned terrainLevel;
public:
    GroundRobot(const std::string& id, unsigned battery, unsigned terrainLevel)
        : Robot(id, battery), terrainLevel(terrainLevel) {}
    bool canExecute(const Mission& m) const {
        return battery >= 2 * m.difficulty &&
            (m.type == MissionType::Patrol || m.type == MissionType::Rescue) &&
            terrainLevel >= m.difficulty; }
    /* за улеснение приемете, че new успешно заделя обекта нявсякъде в програмата */
    GroundRobot* clone() const { return new GroundRobot(*this); }
};

class Drone : public Robot {
    unsigned maxAltitude;
public:
    Drone(const std::string& id, unsigned battery, unsigned maxAltitude)
        : Robot(id, battery), maxAltitude(maxAltitude) {}
    bool canExecute(const Mission& m) const {
        return battery >= 2 * m.difficulty &&
            (m.type == MissionType::Surveillance || m.type == MissionType::Rescue) &&
            maxAltitude >= m.difficulty; }
    Drone* clone() const { return new Drone(*this); }
};

class Fleet {
    std::vector<Robot*> robots;
public:
    Fleet() = default;
    ~Fleet() {
        for (Robot* r : robots) delete r; }
    Fleet(const Fleet& other) {
```

```
        for (const Robot* r : other.robots) robots.push_back(r->clone()); }
Fleet& operator=(const Fleet& other) {
    if (this != &other) {
        for (Robot* r : robots) delete r;
        robots.clear();
        for (const Robot* r : other.robots) robots.push_back(r->clone());
    }
    return *this; }
Fleet& add(const Robot& r) { robots.push_back(r.clone()); return *this; }
void printAll(std::ostream& os) const {
    for (const Robot* r : robots) { r->print(os); os << "\n"; } }
int assignToAll(const Mission& m) {
    int count = 0;
    for (Robot* r : robots) { if (r->assign(m)) ++count; }
    return count;
}
};
int main() {
    Fleet fleet;
    fleet.add(GroundRobot("G1", 100, 5))
        .add(Drone("D1", 100, 8))
        .printAll(std::cout);
    std::cout << fleet.assignToAll({MissionType::Rescue, 4}) << "\n";
    return 0;
}
```

Критерии за оценяване

12 точки. Максимални точки по класове:

- Коректни Mission и Robot: 4т. (9 полета)
- Коректен GroundRobot: 2т. (4 полета)
- Коректен Drone: 2т. (5 полета)
- Коректен Fleet: 4т. (9 полета)

За всяко вярно попълнено празно поле се добавят 0.5 точки до достигане на максимума в рамките на посочената част от програмата. Т.е. напълно верен Drone носи 2 точки.

Задача 2.

Софтуерна система за обучение позволява на потребителите да упражняват техниката си на шофиране чрез игра със симулиране на уличната обстановка в голям град. За системата е формулирано следното изискване:

REQ: Само регистриран потребител може да избере валиден маршрут за симулация. Той може да изиграе маршрута след като е заплатил за играта. Системата поддържа два начина на плащане – с банкова карта и чрез система за електронно плащане. Системата дава точки на потребителя в зависимост от представянето в изиграната симулация.

Представете софтуерното изискване с Диаграма на последователностите.

Примерно решение

TODO

Критерии за оценяване

За вярно и пълно записани актьори, компоненти и стъпките на взаимодействието се оценява с 12 точки. При липса на съответен елемент точките се намаляват пропорционално.

Задача 3.

Вашата фирма е сключила договор с клиент за разработване на система за видео наблюдение и контрол на трафика в Бургас. Моля, подгответе план за работа:

- а) Дефинирайте работни пакети и посочете задачите в тях като внимавате всяка задача да води до специфичен краен резултат, който може да се валидира и контролира от клиента.
- б) За всеки работен пакет определете какви материални и нематериални ресурси и експертиза са необходими (директни разходи на фирмата).

Примерно решение**а) Работни пакети (примерно разпределение):**

РП1 Анализ на областта: Проучване на спецификата на системите за видео наблюдение и контрол на трафика; Проучване на пътната мрежа и телекомуникационната свързаност в Бургас; Формулиране на изисквания към системата – телекомуникационна свързаност, хардуер и софтуер

РП2 Софтуер и управление на системата: Проектиране на софтуерната част на системата; Програмиране и тестване на софтуера; Попълване на базата данни за контрол на трафика; Инсталиране на софтуера в центъра за управление

РП3 Хардуер и интеграция: Проектиране на разположението на устройства за видеонаблюдение и телекомуникационна свързаност; Закупуване на материални активи (напр. сървъри, компютри за центъра за управление; камери за видео наблюдение, мрежово оборудване, др. хардуерни компоненти); Инсталиране на хардуера и телекомуникационно свързване; Интеграция и тестване на системата (хардуерна и софтуерна част)

РП4 Комуникация с клиента: Валидиране на изискванията към системата; Валидиране на проекта на системата; Обучение на клиента

РП5 Управление на проекта: Планиране и контрол; Финансово управление; Среци и комуникация на ръководния екип; Контрол на риска и качеството; Отчетност и документация

б)

РП1: експерти по видеонаблюдение, мрежови инженери, софтуерни инженери, експерти по управление на трафика; (разходи за среци, транспорт)

РП2: софтуерни инженери (може да изброяват: архитекти, дизайнери, програмисти, тестери и др.), експерти по управление на трафика; център за данни; (разходи за среци, транспорт)

РП3: експерти по видеонаблюдение, мрежови инженери, програмисти; устройства за центъра за управление (сървъри, компютри и др.); камери за видео наблюдение, мрежово оборудване, др. хардуерни компоненти; разходи за телекомуникационно свързване; (разходи за среци, транспорт)

РП4: организатори, лектори, зали с мултимедия, кетъринг, материали за участниците, офис консумативи

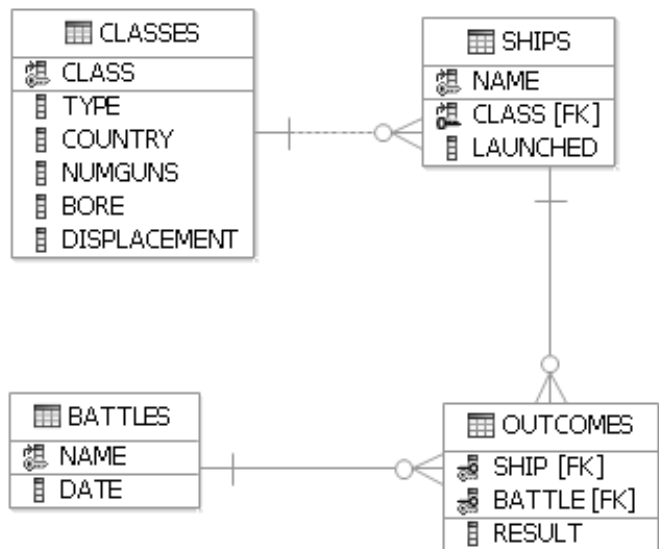
РП5: мениджър, финансист, административен персонал, офис консумативи

Критерии за оценяване

1. Максимален брой точки (8) се дават при планиране на задачи, обхващащи всички етапи на създаване на системата и компонентите ѝ, както и за управлението и комуникация. Разпределението на работните пакети може да следва цикъла на разработката, а не да е разделено по експертиза. За липсата на задачи се намалява оценката с 0,5 за всяка отделна задача (без РП5).
2. Максимален брой точки (4) се дават при представяне на необходимата експертиза, материални активи и разходи за комуникации и мениджмънт. Оценката се намалява с 0,5 за всеки пропуск.

Задача 4.

Дадена е базата от данни Ships. В нея се съхранява информация за кораби (*Ships*) и тяхното участие в битки (*Battles*) по време на Втората световна война. Всеки кораб е построен по определен стереотип, определящ класа на кораба (*Classes*).



Таблицата Classes съдържа информация за класовете кораби:

- class – име на класа, първичен ключ;
- type – тип ('bb' за бойни кораби и 'bc' за бойни крайцери);
- country – държавата, която строи такива кораби;

1) Да се напише SELECT заявка, която за всеки кораб, който има по-малка водоизместимост от средната водоизместимост на класовете на същата държава като неговата, извежда следната информация:

- име на кораба;
- година на пускане на вода;
- брой участия в битки на кораба (или числото 0, ако корабът не е участвал в нито една битка).

Резултатът да бъде сортиран така, че първо да се извеждат най-новите кораби.

2) Дадена е релацията R със следната схема: $R(A, B, C, D)$. Домейните на атрибутите са атомарни множества. В сила са следните функционални зависимости:

$$AB \rightarrow C$$

$$AB \rightarrow D$$

$$C \rightarrow B$$

Намерете всички множества от атрибути, които са ключове на R . Обосновайте решението си.

- numGuns – броят на основните оръдия;
- bore – калибъра им (диаметърът на отвората на оръдието в инчове);
- displacement – водоизместимост (в тонове).

Таблицата Ships съдържа информация за корабите:

- name – име на кораб, първичен ключ;
- class – име на неговия клас;
- launched – годината, в която корабът е пуснат на вода.

Таблицата Battles съдържа информация за битките:

- name – име на битката, първичен ключ;
- date – дата на провеждане.

Таблицата Outcomes съдържа информация за резултатата от участието на даден кораб в дадена битка. Колоните ship и battle заедно формират първичния ключ.

- ship – име на кораба;
- battle – име на битката;
- result – резултат (потънал – 'sunk', повреден – 'damaged', победил – 'ok').

Примерно решение

1)

```
SELECT name, launched, COUNT(battle) AS battles
FROM Ships s
JOIN Classes c ON c.class = s.class
LEFT JOIN Outcomes o ON s.name = o.ship
WHERE displacement < (SELECT AVG(displacement)
                      FROM Classes
                      WHERE country = c.country)
GROUP BY name, launched
ORDER BY launched DESC;
```

2)

Тъй като атрибутът A не присъства в дясната страна на нито една функционална зависимост (ΦZ), то всеки ключ съдържа A .

Тъй като атрибутът D не присъства в лявата страна на нито една ΦZ , но присъства в дясната страна на една от тях, то нито един ключ не съдържа D .

Следователно трябва последователно да проверим за всяко от множествата $\{A\}$, $\{A, B\}$, $\{A, C\}$ и $\{A, B, C\}$. Множеството от атрибути X е суперключ на R , ако $X^+ = \{A, B, C, D\}$. X е ключ, ако е суперключ и никое негово собствено подмножество не е суперключ.

$\{A\}$ не е суперключ, тъй като $\{A\}^+ = \{A\}$, следователно не е и ключ.

$\{A, B\}^+ = \{A, B, C, D\}$, т.е. $\{A, B\}$ е суперключ. Вече беше доказано, че $\{A\}$ не е ключ, аналогично и $\{B\}$ не е ключ, понеже $\{B\}^+ = \{B\}$. Следователно $\{A, B\}$ е ключ.

Аналогично намираме $\{A, C\}^+ = \{A, B, C, D\}$ и показваме, че $\{C\}^+ = \{B, C\}$, т.е. $\{A, C\}$ също е ключ.

След като $\{A, B\}$ е ключ, то $\{A, B, C\}$ е суперключ, но не е ключ.

Критерии за оценяване

1) Коректно решение носи 6 т. За всяка сгрешена клауза броят на точките се намалява с 2 до достигане на 0 т.

2) Посочването на верните отговори носи до 2 т., а доказателството им носи още до 4 т. по следната схема:

Всички ключове на релацията са $\{A, B\}$ и $\{A, C\}$. Посочването на всеки от тези отговори носи по 1 т., а при посочването на грешно множество от атрибути броят на точките се намалява с по 1 за всеки грешен отговор до достигане на 0 т.

Коректно доказателство за всеки от двата ключа, използващо обвивки на атрибути или аксиоми на Армстронг, носи по 2 точки.

Задача 5.

Спрямо ортонормирана координатна система $K = Oxy$ в равнината е дадена точката

$$B(8, 3)$$

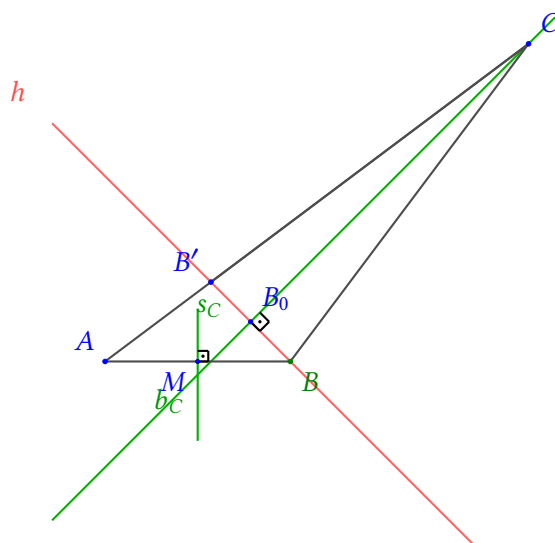
и правите

$$b_C : x - y - 2 = 0, \quad s_C : 2x - 9 = 0,$$

които са съответно вътрешната ъглополовяща при върха C и симетралата на страната AB на $\triangle ABC$.

- а) Да се намерят координатите на точката B' , която е ортогонално симетрична на точката B спрямо правата b_C ;
- б) Да се намерят координатите на върховете A и C на $\triangle ABC$;
- в) Да се намери лицето на $\triangle ABC$.

Примерно решение



а)

Търсим точката $B'(x', y')$, която е симетрична на $B(8, 3)$ спрямо правата

$$b_C : x - y - 2 = 0.$$

Построяваме правата h от условията, че е перпендикулярна на правата b_C и минава през точката B :

$$h : x + y - 11 = 0.$$

Нека с $B_0(x_0, y_0)$ означим ортогоналната проекция на точката B върху правата b_C . B_0 е пресечна точка на правите b_C и h . Следователно нейните координати удовлетворяват

$$\begin{cases} x - y - 2 = 0, \\ x + y - 11 = 0. \end{cases}$$

Оттук намираме, че

$$B_0 \left(\frac{13}{2}, \frac{9}{2} \right).$$

От друга страна B_0 е среда на отсечката BB' и от това, че

$$x_0 = \frac{x_B + x'}{2}, \quad y_0 = \frac{y_B + y'}{2},$$

получаваме

$$\frac{13}{2} = \frac{8 + x'}{2}, \quad \frac{9}{2} = \frac{3 + y'}{2}.$$

Следователно

$$\boxed{B'(5, 6)}.$$

б)

Тъй като s_C е симетрала на страната AB , то точките A и B са симетрични спрямо правата

$$s_C : 2x - 9 = 0.$$

Намираме уравнението на правата c от условията, че е перпендикулярна на правата s_C и минава през точката B :

$$c : y - 3 = 0.$$

Нека с $M(x_M, y_M)$ означим ортогоналната проекция на точката B върху правата s_C . M е пресечна точка на правите s_C и s . Следователно намираме, че

$$M\left(\frac{9}{2}, 3\right).$$

От друга страна M е среда на отсечката AB и от това, че

$$x_M = \frac{x_A + x_B}{2}, \quad y_M = \frac{y_A + y_B}{2},$$

получаваме

$$\frac{9}{2} = \frac{x_A + 8}{2}, \quad 3 = \frac{y_A + 3}{2}.$$

Следователно

$$\boxed{A(1, 3)}.$$

Понеже b_C е вътрешна ъглополовяща при върха C , правите CB и CA са симетрични спрямо b_C . Следователно точките A , C и B' лежат на една права.

Намираме правата AC от условието, че съдържа точките A и B' :

$$AC : \begin{vmatrix} x & y & 1 \\ 1 & 3 & 1 \\ 5 & 6 & 1 \end{vmatrix} = 0.$$

Получаваме

$$AC : 3x - 4y + 9 = 0.$$

Точката C лежи едновременно на AC и на b_C , затова

$$\begin{cases} 3x - 4y + 9 = 0, \\ x - y - 2 = 0. \end{cases}$$

Следователно

$$\boxed{C(17, 15)}.$$

в)

Лицето на триъгълника ABC намираме по формулата

$$S_{\triangle ABC} = \frac{1}{2} \left\| \begin{vmatrix} 1 & 3 & 1 \\ 8 & 3 & 1 \\ 17 & 15 & 1 \end{vmatrix} \right\| = 42.$$

$$\boxed{S_{\triangle ABC} = 42}.$$

Критерии за оценяване

- а) – за намиране на правата h - 2 т.;
– за намиране на точката B_0 - 1 т.;
– за намиране на точката B' - 1 т.
- б) – за намиране на точката A - 2 т.;
– за намиране на правата AC - 2 т.;
– за намиране на точката C - 2 т.
- в) – за пресмятане на лицето на триъгълника ABC - 2 т.

Задача 6.

Фирма произвежда монитори, като всеки от тях се състои от един милион пиксела. В мониторите II качество средният брой на дефектните пиксели е три, а в мониторите I качество има средно един дефектен пиксел. Считаме, че за всеки пиксел в монитор I качество, вероятността да е дефектен е една и съща. Същото важи и за мониторите II качество. Също така, дефектирането на пиксел е независимо събитие от дефектирането на който и да е друг пиксел.

- а) На колко е равна вероятността пиксел в монитор I качество да е дефектен?
 - б) Каква е вероятността в монитор от I качество да има по-малко от три дефектни пиксела?
 - в) За мониторите от I или от II качество е по-вероятно да имат точно два дефектни пиксела?
 - г) Ако 90% от мониторите са I качество, а останалите – II, каква е вероятността случайно избран монитор да има точно два дефектни пиксела.
-

Примерно решение

а) Нека X е броят на дефектните пиксели в монитор I качество и p е търсената вероятност. Всички пиксели са $n = 1\,000\,000$, $X \in Bi(n, p)$. Знаем математическото очакване $EX = 1$. От друга страна при биномно разпределение $EX = np$, следователно $p = 0.000001$.

б) При голям брой на опитите n и малка вероятност за успех p , за пресмятане на броя на успехите се използва Пуасоново разпределение с параметър $\lambda = np = 1$ и вероятностите се пресмятат по формулата $P(X = k) = \frac{\lambda^k e^{-\lambda}}{k!}$, $k = 0, 1, 2, \dots$. В нашия случай $X \in Po(1)$, следователно

$$P(X < 3) = P(X = 0) + P(X = 1) + P(X = 2) = e^{-1} + e^{-1} + \frac{1^2 e^{-1}}{2!} = \frac{5}{2} e^{-1}$$

в) Нека Y броят на дефектните пиксели в монитор II качество, тогава $Y \in Po(3)$ и $P(Y = 2) = \frac{3^2 e^{-3}}{2!}$. Следните неравенства са еквивалентни

$$\begin{aligned} P(X = 2) &< P(Y = 2) \\ \frac{1}{2} e^{-1} &< \frac{9}{2} e^{-3} \\ e^2 &< 9 \end{aligned}$$

Последното неравенство е очевидно, т.е. за мониторите II качество е по-вероятно да имат два дефектни пиксела.

г) Използваме формула за пълна вероятност с хипотези {избран е монитор I качество}, {избран е монитор II качество}. Търсената вероятност е

$$0.9P(X = 2) + 0.1P(Y = 2) = 0.9 \frac{1^2 e^{-1}}{2!} + 0.1 \frac{3^2 e^{-3}}{2!} = \frac{9}{20} (e^{-1} + e^{-3})$$

Критерии за оценяване

За напълно коректни решения на съответните подточки:

- а) максимум 2 точки
- б) максимум 3 точки
- в) максимум 4 точки
- г) максимум 3 точки

Чернова